

Course Module: Convolutional Neural Networks

Latreche Sara

Contents

1	Introduction	4
2	Building a CNN	9
2.1	Motivation & Intuition	9
2.2	Image Filters and Convolutions	9
2.3	Edges and Edge-Detection Filters	11
2.4	Filter Banks and Tensor Representations	14
2.5	From Filter Banks to Channels	15
2.6	Max Pooling Layers	20
2.7	Typical CNN Architecture	23
2.8	Backpropagation in a Simple CNN	24
3	Case Studies	26
3.1	Why Look at Case Studies?	26
3.2	Outline of Case Studies	27
3.3	Classic Networks	27
3.4	LeNet-5	27
3.5	AlexNet: A Turning Point in Deep Learning	28
3.5.1	The ImageNet Competition	28
3.5.2	Why AlexNet Was Revolutionary	29
3.5.3	The AlexNet Architecture	29

3.5.4	Impact	29
3.5.5	Inspiration	30
3.6	VGGNet (VGG-16)	32
3.7	Residual Networks (ResNets)	34
Active Recall: CNN Architectures		37
4	From Image Classification to Computer Vision Tasks	39
4.1	Image Classification vs Object Detection	39
4.2	Segmentation	40
4.3	Multimodal Understanding	40
4.4	Pose Estimation and Keypoint Detection	42
4.5	Object Tracking	43
4.6	Object Counting	44
5	YOLO:You only look once	46
5.1	Abstract and Introduction: Core Idea	46
5.2	Unified Detection Formulation	46
5.3	Network Architecture (High-Level)	47
5.4	Loss Function	48
5.5	Limitations of YOLO	49
6	A Picture is Worth 16×16 Patches: From CNNs to Vision Transformers	51
6.1	Vision Transformer (ViT) Architecture	51
6.2	Comparison Between CNNs and Vision Transformers	52
7	Conclusion	53

“Shadow is the means by which bodies display their form.”

— Leonardo da Vinci, *Treatise on Painting*

1 Introduction

The ambition to endow machines with the ability to interpret visual information extends far beyond the emergence of modern deep learning. Vision, for a machine, is not merely the processing of pixel intensities, but the extraction of structured information such as shapes, textures, objects, and spatial relationships. Understanding how biological systems achieve this transformation—from light stimuli at the retina to semantic perception—has long inspired both neuroscientists and computer scientists.

Historical and Neuroscience Perspective

Convolutional Neural Networks (CNNs) are not purely mathematical inventions; they are deeply rooted in decades of interdisciplinary research spanning neuroscience and computer vision. They reflect an ongoing scientific effort to understand **how hierarchical perception emerges in the brain**.

A major conceptual breakthrough emerged from the seminal work of **David Hubel and Torsten Wiesel** in the 1960s. Through neurophysiological experiments on the visual cortex, they demonstrated that neurons respond selectively to oriented edges and specific spatial patterns. Their findings revealed a hierarchical organization of visual processing: **simple cells** detect elementary features, while **complex cells** combine these responses to represent increasingly abstract structures. This discovery, later recognized with the Nobel Prize in 1981 [1], provided the biological intuition underlying convolutional feature extraction.

Building upon these biological principles, **Kunihiko Fukushima** introduced the *Neocognitron* in 1980 [2], one of the earliest computational models to implement hierarchical feature detection. Although limited by the computational resources of its time, it established the architectural blueprint of alternating feature-extraction and pooling stages.

The decisive advancement came in 1998 when **Yann LeCun** and collaborators integrated backpropagation into such architectures, giving rise to **LeNet-5** [3]. Unlike earlier hand-crafted systems, LeNet-5 learned its convolutional filters directly from data, demonstrating the practical power of end-to-end trainable feature hierarchies for handwritten digit recognition.

After a period of limited progress due to data and hardware constraints, CNNs experienced a dramatic resurgence in 2012 with **AlexNet** [4]. Leveraging GPU acceleration and the large-scale **ImageNet** dataset, AlexNet significantly outperformed traditional computer vision methods. This milestone marked the beginning of the deep learning revolution in visual recognition.

Subsequently, research expanded beyond classification toward more complex tasks such as **Object Detection** and **Face Recognition**. A notable milestone was the introduction of **YOLO (You Only Look Once)** by **Joseph Redmon** in 2016 [5], which reformulated object detection as a single regression problem, enabling real-time performance in practical applications including surveillance and robotics.

More recently, the field has witnessed another paradigm shift with the integration of attention mechanisms into vision models. The **Vision Transformer (ViT)** [6] demonstrated that images can be partitioned into fixed-size patches and processed using self-attention mechanisms to capture long-range dependencies. This approach challenges the locality bias inherent in convolutional architectures and has paved the way for transformer-based and multimodal models.

Today, visual understanding extends beyond recognition to reasoning. Multimodal systems such as CLIP and GPT-4o integrate visual and linguistic representations, moving toward a unified framework in which machines not only perceive images but also interpret and contextualize them.

Neuroscience Foundations (1960s)

In the 1960s, **David Hubel and Torsten Wiesel** showed that neurons in the visual cortex respond selectively to edges and orientations. They discovered:

- **Simple cells:** activated by specific edge orientations in fixed positions.
- **Complex cells:** activated by edges regardless of exact position.

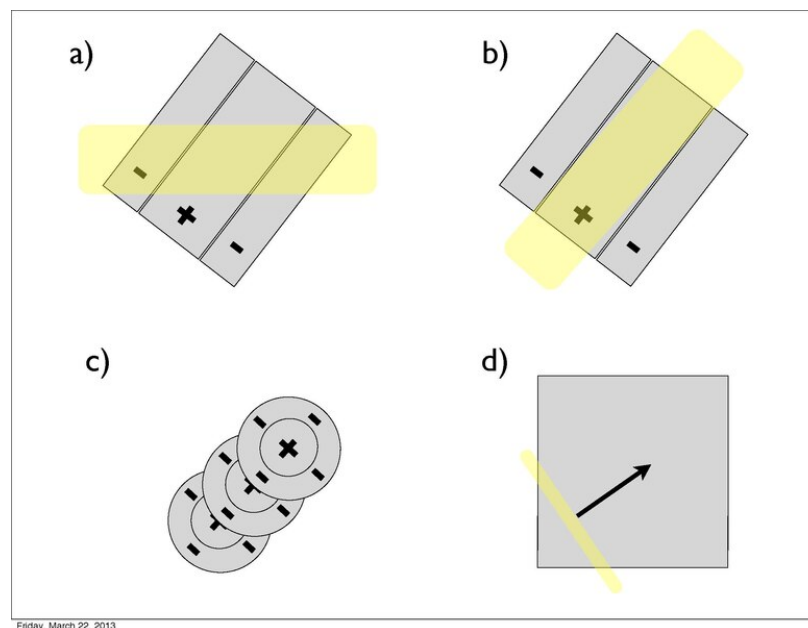


Figure 1: Hierarchical organization of the visual cortex: simple cells (edges) feed into complex cells (invariance). Source: Wikimedia Commons, CC-BY-SA.

Hubel & Wiesel's work earned the **1981 Nobel Prize in Physiology or Medicine** and still inspires CNN design today.

Hand-crafted Filters (1970s–1980s)

Inspired by these biological insights, researchers created filters that mimic edge detection: - **Sobel operator (1968)** - **Prewitt operator (1970s)** - **Gabor filters (1980s)**

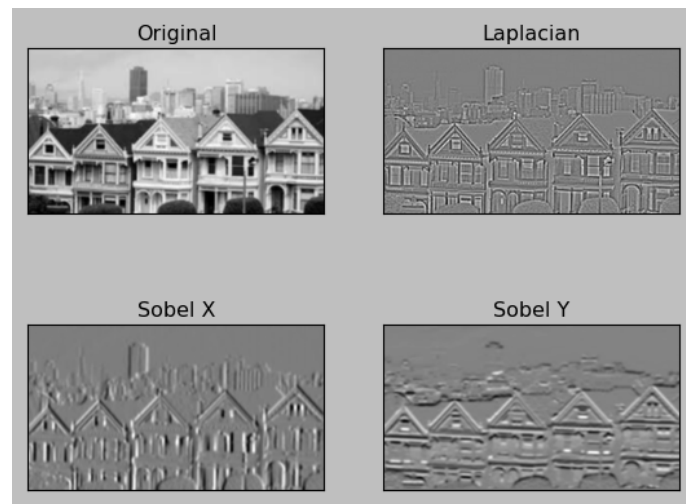


Figure 2: Sobel operator applied to an image: left = original, right = edge map.

Birth of CNNs (1980–1998)

Kunihiko Fukushima proposed the *Neocognitron* (1980), introducing: - Local receptive fields, - Weight sharing, - Subsampling (early pooling).

Yann LeCun later implemented these ideas in *LeNet-5* (1998), used for handwritten digit recognition (MNIST).

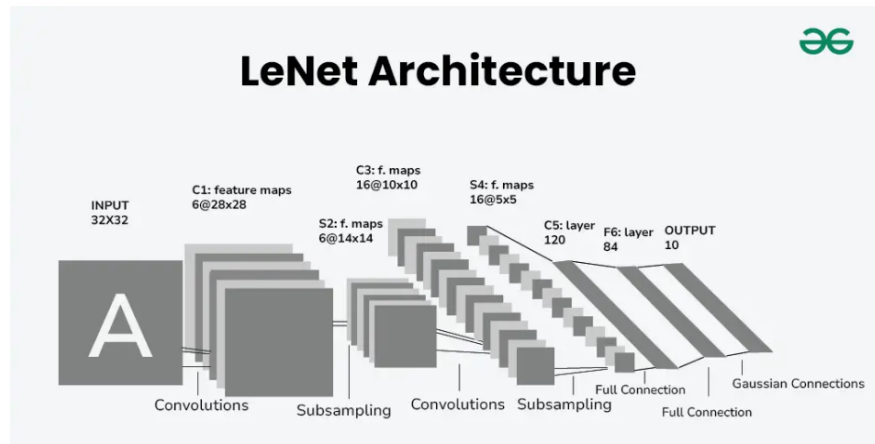


Figure 3: Architecture of LeNet-5, one of the first CNNs successfully applied to handwritten digit recognition. Source: LeCun et al. (1998) [Link](#)

The Deep Learning Revolution (2012)

CNNs became mainstream after the success of **AlexNet** (Krizhevsky et al., 2012), which won the ImageNet competition by a huge margin.

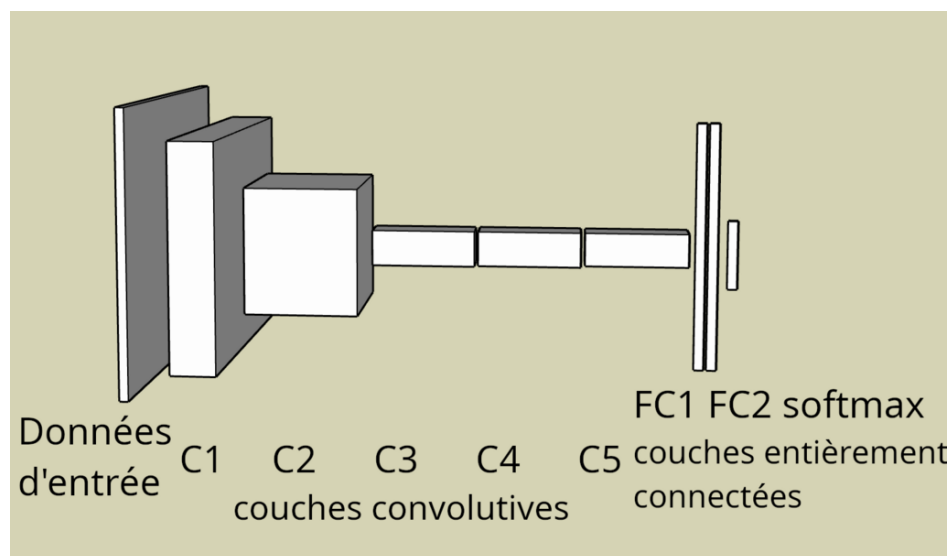


Figure 4: AlexNet architecture, which revolutionized computer vision in 2012. Source: Krizhevsky et al. (2012), ImageNet paper. [Link](#)

Timeline of CNN and Computer Vision History

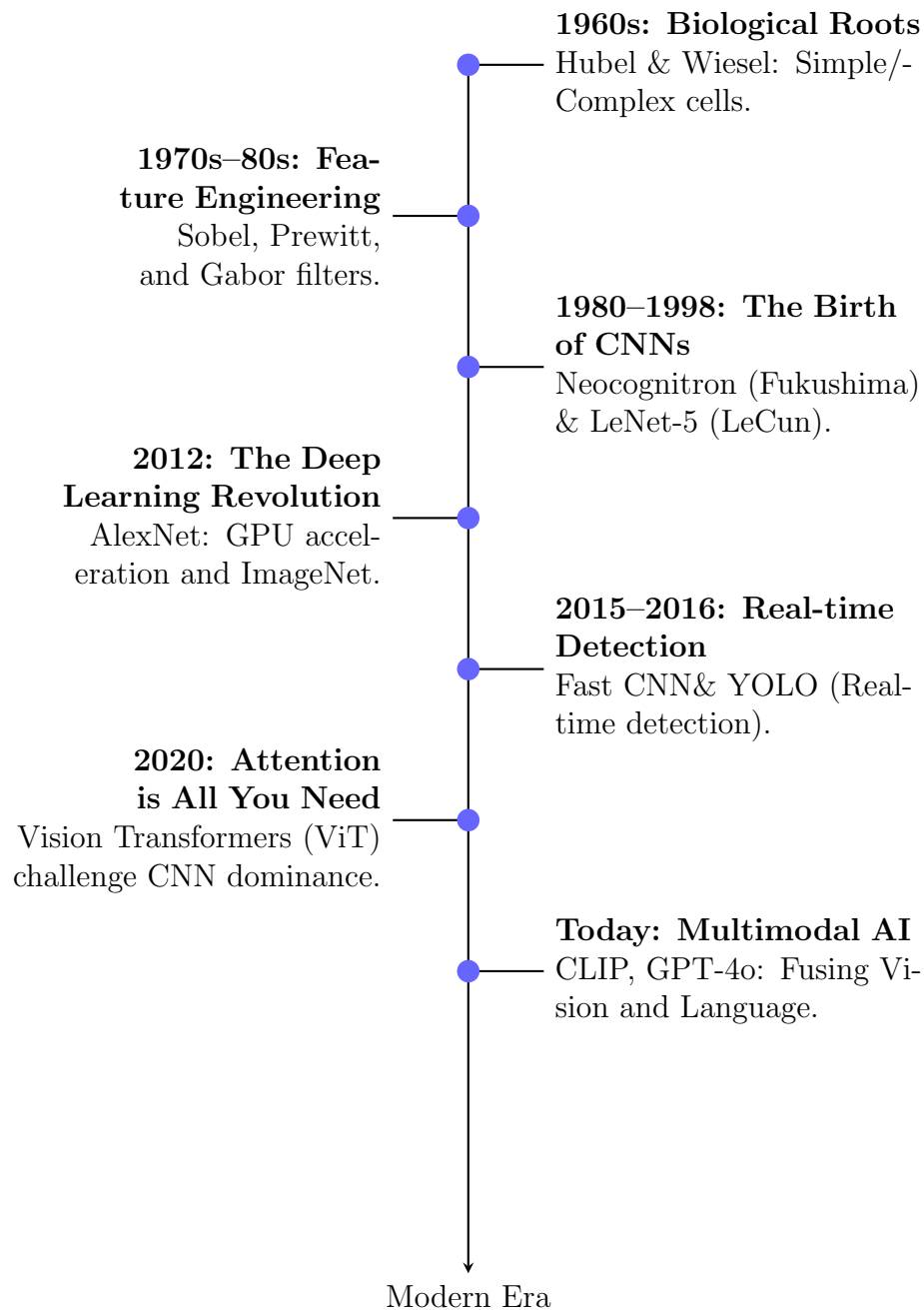


Figure 5: Timeline of the evolution from biological vision to modern multimodal transformers.

2 Building a CNN

2.1 Motivation & Intuition

Note

Traditional fully connected networks treat every pixel equally, which is inefficient for images. CNNs solve this by leveraging spatial structure.

Two key insights:

- **Locality:** relevant features (edges, textures) are local in space.
- **Translation invariance:** the same feature can appear anywhere.

Applications: computer vision, speech, medical imaging, etc.

2.2 Image Filters and Convolutions

Note

An **image filter** is a function that scans a local neighborhood of pixels and detects whether a certain pattern (such as an edge or a corner) is present in that region.

1D Example: Binary Image

To build intuition, let us start with a simple case: a **1-dimensional binary image** (a row of black/white pixels) and a filter F of size two.

The filter is a short vector of weights that we slide across the image. At each position, we compute the dot product between the filter and the corresponding local patch of pixels.

Definition

If X is the input image of length d , the output image Y is given by:

$$Y_i = F \cdot (X_{i-1}, X_i)$$

To ensure the output has the same size as the input, we usually add zeros at the borders of X . This is called **padding**.

Concrete Example

Let the filter be $F_1 = (-1, +1)$. When we convolve it with the 1D binary image shown below, it produces a new sequence that highlights positions where a transition from 0 to 1 occurs.

Example	
Input image (1D binary pixels):	$X = [0, 0, 1, 1, 1, 0, 1, 0, 0, 0]$
Filter:	$F_1 = [-1, +1]$
Output after convolution:	$Y = [0, 1, 0, 0, -1, 1, -1, 0, 0]$

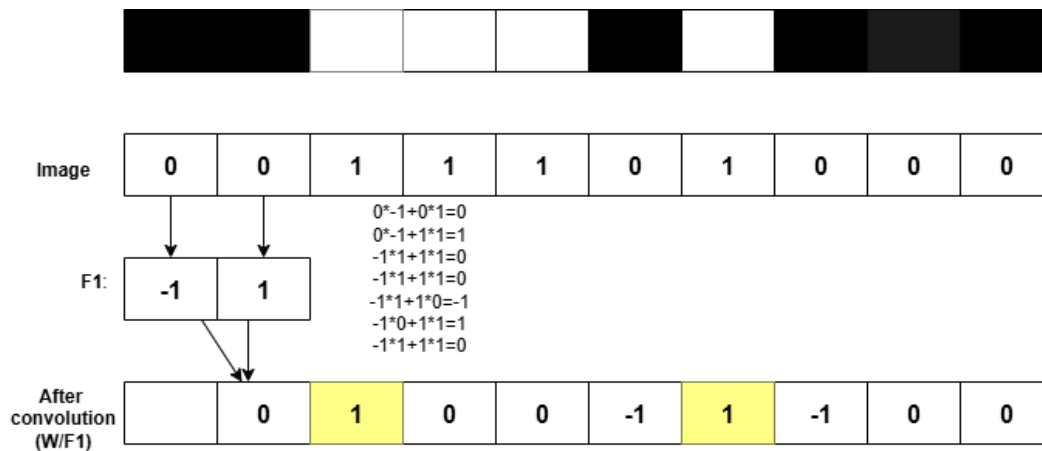


Figure 6: Applying filter $F_1 = [-1, +1]$ to a 1D binary image. The highlighted positive responses correspond to left edges in the image (transitions $0 \rightarrow 1$).

Another interesting filter is $F_2 = (-1, +1, -1)$. When applied to the same binary image, F_2 responds strongly to **isolated bright pixels**, i.e., when a '1' is surrounded by '0's.

Second Example: Isolated Pixel Detection

Now let us try a different filter:

$$F_2 = [-1, +1, -1]$$

This filter examines three consecutive pixels at a time. It responds strongly only when the

central pixel is bright (1) and both its neighbors are dark (0). In other words, it acts as a **detector for isolated positive pixels**.

Example

Input image:

$$X = [0, 0, 1, 1, 1, 0, 1, 0, 0, 0]$$

Filter:

$$F_2 = [-1, +1, -1]$$

Output after convolution:

$$Y = [-1, 0, -1, 0, -2, 1, -1, 0]$$

Image	0	0	1	1	1	0	1	0	0	0
F1:	-1	1	1							
After convolution (W/F2)		-1	0	-1	0	-2	1	-1	0	

Figure 7: Applying filter $F_2 = [-1, +1, -1]$ to a 1D binary image. Notice how the positive response (+1, highlighted) occurs when the central pixel is isolated (a 1 surrounded by 0s).

Study Question

Why does F_2 ignore regions with multiple consecutive 1s but respond to a single isolated 1?

2.3 Edges and Edge-Detection Filters

Definition

Edge: In image processing, an edge is a region in the image where the intensity (or color) changes sharply. Edges correspond to object boundaries, texture changes, or lighting transitions, and they are fundamental features used by CNNs to build higher-level representations.

Directional Edge Filters

Just as in our 1D examples, filters in 2D can be designed to detect intensity changes in specific directions:

- **Left Edge Filter:** Detects transitions from dark to bright along the horizontal axis.

$$K = \begin{bmatrix} -1 & 1 \end{bmatrix}$$

- **Right Edge Filter:** Detects transitions from bright to dark along the horizontal axis.

$$K = \begin{bmatrix} 1 & -1 \end{bmatrix}$$

- **Top Edge Filter:** Detects transitions from dark to bright vertically.

$$K = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

- **Bottom Edge Filter:** Detects transitions from bright to dark vertically.

$$K = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Prewitt Operator

The **Prewitt operator** uses 3×3 kernels that approximate the derivative in horizontal and vertical directions.

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad K_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

- K_x : detects vertical edges (left vs. right). - K_y : detects horizontal edges (top vs. bottom).

Note

Variants such as Prewitt-Top, Prewitt-Bottom, Prewitt-Left, and Prewitt-Right emphasize edges in specific directions by modifying the weight distribution.

Sobel Operator

The **Sobel operator** is similar to Prewitt but gives more weight to the center row or column, making it more robust to noise.

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad K_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

- K_x : detects vertical edges. - K_y : detects horizontal edges.

Key Insight

Edge filters are the building blocks of convolutional neural networks. - Early CNN layers often learn filters resembling Prewitt or Sobel operators. - Higher layers build on these edges to detect shapes, textures, and objects.

Think About It

Why would a CNN prefer to learn edge filters automatically instead of using fixed ones like Sobel or Prewitt?

Convolution vs. Correlation

In classical signal processing, convolution and correlation are distinct operations. In deep learning, however, most libraries implement the correlation operation but still refer to it as **convolution**.

Note

This naming difference does not affect learning: CNNs can discover the correct filters regardless of whether the operation is technically convolution or correlation.

Concrete Example

Let the filter be $F_1 = (-1, +1)$. Sliding this filter across a binary 1D image highlights positions where a transition from 0 to 1 occurs — effectively acting as a **left-edge detector**.

Example

Input image (binary row of pixels):

$$X = [0, 0, 1, 1, 1, 0, 1, 0, 0]$$

Filtered output:

$$Y = [0, 1, 0, 0, -1, 1, -1, 0, 0]$$

Another filter, $F_2 = (-1, +1, -1)$, responds strongly to isolated bright pixels (detecting small “dots” in the binary image).

Interpretation

- Filters act as **pattern detectors**.
- Different filters highlight different aspects of the image (edges, dots, textures).
- The result of applying a filter is called a **feature map**.

Hands-on Lab

Hands-on: Implement a 1D convolution in Python.

```
import numpy as np

# Input 1D binary "image"
X = np.array([0,0,1,1,1,0,1,0,0,0])

# Define filter
F = np.array([-1, 1])

# Convolve manually
Y = []
for i in range(1, len(X)):
    patch = X[i-1:i+1]    # local neighborhood
    Y.append(np.dot(F, patch))

print("Output:", Y)
```

Try replacing F with $[-1, 1, -1]$ and observe the result.

Hands-on Lab

Implement a 3×3 filter in NumPy and apply it to a grayscale image. Visualize edge detection.

2.4 Filter Banks and Tensor Representations

Note

Two-dimensional filters similar to those we have seen are thought to exist in the **visual cortex** of all mammalian brains. Statistical analysis of natural images also shows that edge-like patterns occur frequently, which may explain why such filters are biologically relevant.

2.5 From Filter Banks to Channels

Early computer vision researchers often designed **filter banks** by hand. A **filter bank** is simply a collection of filters applied to the same input image.

Definition

If k filters are applied to an image of size $n \times n$, the result is a stack of k new images, called **channels**, producing an output tensor of shape $n \times n \times k$.

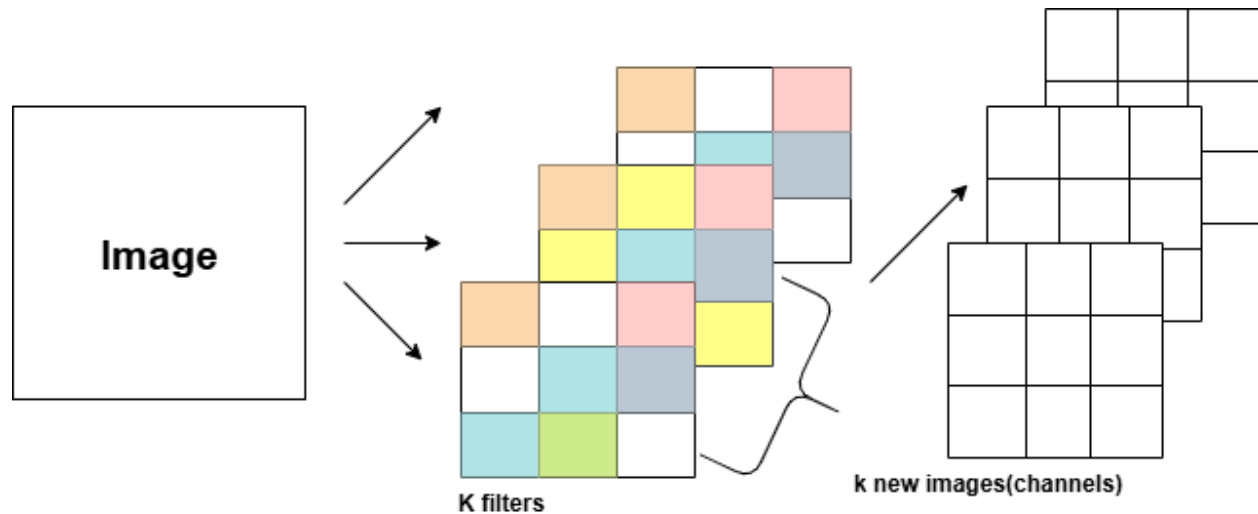


Figure 8: Illustration of a filter bank: multiple filters applied to one input image produce multiple channels, which can be stacked into a 3D tensor. imagine stacking all these new images up so that we have a cube of data, indexed by the original row and column indices of the image, as well as by the channel. The next set of filters in the filter bank will generally be three-dimensional: each one will be applied to a sub-range of the row and column indices of the image and to all of the channels.

Tensors in CNNs

Once we stack multiple channels, we obtain a **tensor** representation of the data. Subsequent filters must therefore be three-dimensional: they span both the *spatial dimensions* (row and column indices) and the *channel dimension*.

Example

Suppose we have two 3×3 filters in the first layer:

$$f_1 = (\text{vertical edge detector}), \quad f_2 = (\text{horizontal edge detector}).$$

- Input: an $n \times n$ grayscale image. - Output: an $n \times n \times 2$ tensor (two channels: vertical and horizontal edges). - Next: a 3D filter can look for patterns like “two horizontal bars + two vertical bars,” producing a single final $n \times n$ feature map.

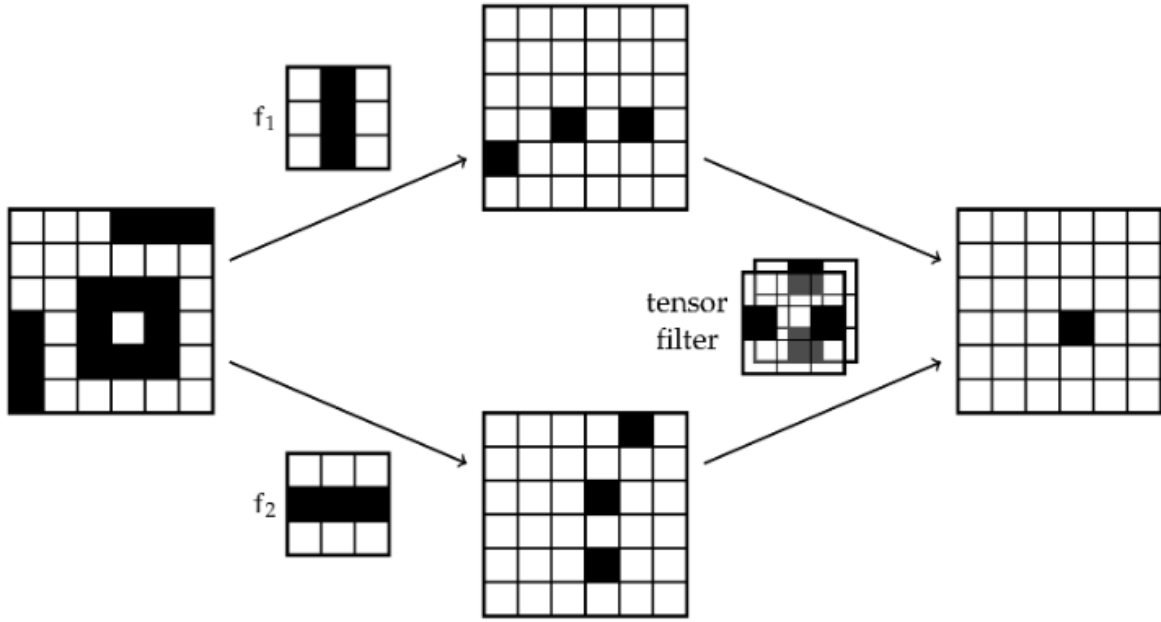


Figure 9: Illustration of two 3×3 filters (vertical and horizontal) producing two feature maps that are stacked into a tensor, followed by a 3D filter extracting combined patterns.

[7]

Formal Definition of a Convolutional Layer

A convolutional layer l can be specified by the following hyperparameters:

- **Number of filters:** m^l .
- **Filter size:** $k^l \times k^l \times m^{l-1}$, where m^{l-1} is the number of input channels.
- **Stride:** s^l , the spacing at which the filter is applied.
- **Input size:** $n^{l-1} \times n^{l-1} \times m^{l-1}$.

The output has shape:

$$n^l \times n^l \times m^l, \quad \text{where } n^l = \left\lfloor \frac{n^{l-1}}{s^l} \right\rfloor.$$

Each filter contains $k^l \times k^l \times m^{l-1}$ weights, and there are m^l filters in total.

Key Insight

Thanks to **weight sharing**, CNNs require far fewer parameters than fully connected networks. The same filter weights are reused across the entire image, capturing translation-invariant features.

Study Questions

1. **How many weights are there in a convolutional layer with filter size $k^l \times k^l \times m^{l-1}$ and m^l filters?**

Each filter has $k^l \times k^l \times m^{l-1}$ weights. Since there are m^l filters, the total number of weights is:

$$m^l \cdot (k^l \times k^l \times m^{l-1}).$$

2. **If we used a fully connected layer with the same input and output dimensions ($n^{l-1} \times n^{l-1} \times m^{l-1}$ inputs, $n^l \times n^l \times m^l$ outputs), how many weights would it have?**

A fully connected layer would have one weight per input–output connection:

$$(n^{l-1} \times n^{l-1} \times m^{l-1}) \cdot (n^l \times n^l \times m^l).$$

3. **Compare the two cases. Why is the convolutional layer so much more parameter-efficient?**

In the convolutional layer, the same small set of weights is *shared* across the entire image. In the fully connected layer, each output unit has its own separate set of weights for every input. Thus, convolution drastically reduces the number of parameters, making training faster, less prone to overfitting, and better at capturing local patterns.

Worked Example: Parameter Counts

Let's compare the two cases for a concrete example:

- Input: $32 \times 32 \times 3$ (a small color image).
- Convolutional layer: filter size 5×5 , $m^l = 6$ filters.

Convolutional layer weights:

$$6 \cdot (5 \times 5 \times 3) = 6 \cdot 75 = 450 \text{ weights.}$$

Fully connected layer weights:

$$(32 \times 32 \times 3) \cdot (32 \times 32 \times 6) = 3072 \cdot 6144 = 18,874,368 \text{ weights.}$$

Comparison: 450 vs. over 18 million! This dramatic difference illustrates why CNNs are vastly more efficient than fully connected networks for image data.

Hands-on Lab: Filters and Channels in PyTorch

Step 1: Load a real image (CIFAR-10 sample).

```
import torch
import torchvision
import torchvision.transforms as T
import matplotlib.pyplot as plt

# Load CIFAR-10 dataset
transform = T.Compose([T.ToTensor()])
dataset = torchvision.datasets.CIFAR10(root="./data", train=False,
                                       download=True, transform=
                                       transform)

img, label = dataset[0]  # one sample (shape: 3x32x32)

# Show the image
plt.imshow(img.permute(1, 2, 0))  # convert CHW -> HWC
plt.title(f"Label: {label}")
plt.show()
```

Step 2: Apply a convolutional layer.

```
import torch.nn as nn

conv = nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5)

# Add batch dimension (1,3,32,32)
x = img.unsqueeze(0)
y = conv(x)

print("Input shape :", x.shape)
print("Output shape:", y.shape)
```

Step 3: Count parameters.

```
def count_params(model):
    return sum(p.numel() for p in model.parameters() if p.
               requires_grad)

print("Convolutional parameters:", count_params(conv))
```

Step 4: Inspect weights.

```
print("Conv weight shape:", conv.weight.shape)
print("Conv bias shape  :", conv.bias.shape)
```

Expected Output (abridged):

```
Input shape : torch.Size([1, 3, 32, 32])
Output shape: torch.Size([1, 6, 28, 28])
Convolutional parameters: 456
Conv weight shape: torch.Size([6, 3, 5, 5])
```

2.6 Max Pooling Layers

Definition

A **max pooling layer** is a downsampling operation that reduces the spatial size of feature maps. It has no trainable weights: instead, it slides a window of size $k \times k$ across the input and outputs the maximum value in each region.

Intuition

- Pooling reduces the resolution of the image, building a **feature pyramid**.
- Early layers detect local edges; later layers pool these into larger, more abstract patterns.
- Max pooling achieves a form of **translation invariance**: the exact location of a feature is less important than its presence.

Note

Max pooling is like “summarizing” an image region by its strongest activation. This helps the network generalize and prevents it from overfitting to tiny pixel-level details.

Example

Consider a $64 \times 64 \times 3$ image. A max pooling layer with stride = $k = 2$ will produce a $32 \times 32 \times 3$ output.

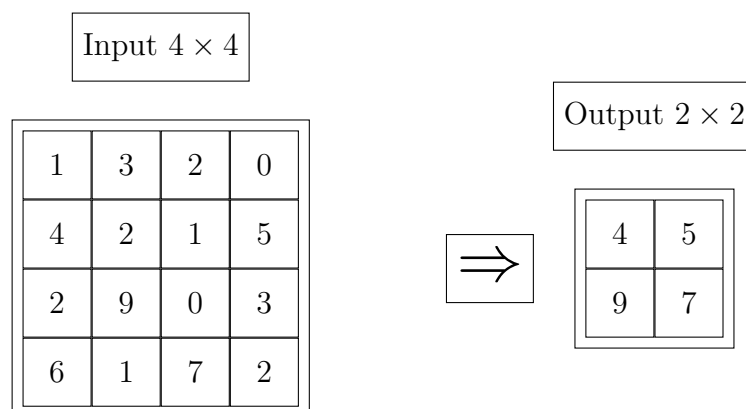


Figure 10: Illustration of max pooling with $k = 2$, stride=2. Each 2×2 block in the input is replaced by its maximum value in the output.

Study Question

Maximilian Poole suggests adding two max pooling layers with size $k = 2$, stride=2, one after the other. What single operation would this be equivalent to?

Example

Answer: It would be equivalent to a single pooling layer with stride = 4. Applying 2×2 pooling twice reduces a 64×64 image first to 32×32 , then to 16×16 , which is the same as pooling with $k = 4$, stride=4 in one step.

Hands-on Lab: Max Pooling in PyTorch

Step 1: Load an image from CIFAR-10.

```
import torch
import torchvision
import torchvision.transforms as T
import matplotlib.pyplot as plt

# Load CIFAR-10 (test set)
transform = T.Compose([T.ToTensor()])
dataset = torchvision.datasets.CIFAR10(root="./data", train=False,
                                       download=True, transform=
                                       transform)

img, label = dataset[0] # shape: (3, 32, 32)
plt.imshow(img.permute(1, 2, 0))
plt.title(f"Original CIFAR-10 sample (label={label})")
plt.show()
```

Step 2: Apply max pooling with $k = 2$, stride=2.

```
import torch.nn as nn

pool = nn.MaxPool2d(kernel_size=2, stride=2)

x = img.unsqueeze(0) # add batch dimension -> (1,3,32,32)
y = pool(x)          # pooled output -> (1,3,16,16)

print("Input shape :", x.shape)
print("Output shape:", y.shape)

# Visualize pooled image
plt.imshow(y.squeeze().permute(1, 2, 0))
plt.title("After Max Pooling (stride=2, k=2)")
plt.show()
```

Expected Output (abridged):

Input shape : torch.Size([1, 3, 32, 32])
Output shape: torch.Size([1, 3, 16, 16])

Discussion:

- The 32×32 image shrinks to 16×16 , while keeping 3 channels.
- Max pooling reduces spatial resolution but preserves the strongest activations.
- This operation builds a feature pyramid: early local details $\rightarrow$ larger patterns in later layers.

2.7 Typical CNN Architecture

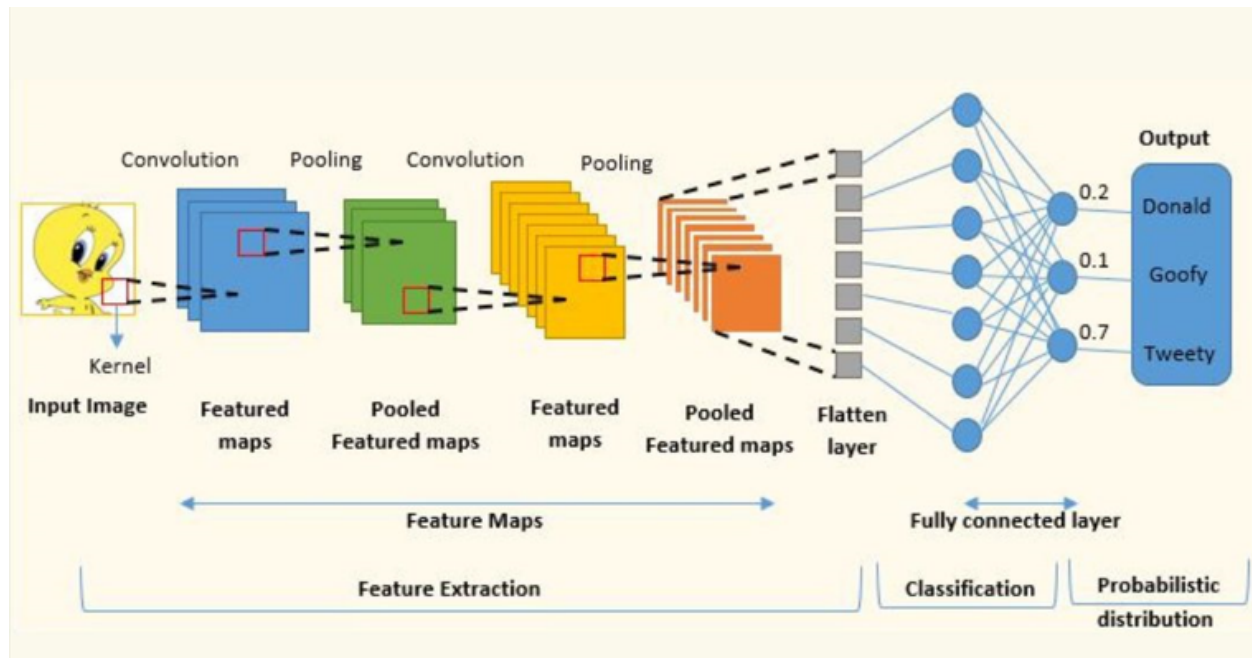


Figure 11: Typical CNN architecture: alternating convolution, activation, and pooling layers, followed by fully connected and output layers. (Adapted from

Definition

A **Convolutional Neural Network (CNN)** typically consists of repeated stacks of convolutional layers (filters), non-linear activations (ReLU), and pooling layers, followed by fully connected layers and an output layer (often softmax for classification).

Layer-by-Layer Structure

1. **Convolution + ReLU:** Detect local patterns (edges, textures). ReLU introduces non-linearity.
2. **Pooling:** Downsample feature maps, summarizing activations and reducing spatial size.
3. **Stacking:** Multiple rounds of Conv+ReLU+Pooling build a hierarchy of features: from edges → shapes → objects.
4. **Fully Connected Layers:** Interpret the learned features as global patterns.
5. **Softmax Output:** Produces class probabilities.

Key Insight

Although CNNs are built from simple blocks (Conv, ReLU, Pooling, FC, Softmax), their composition creates powerful hierarchical representations. Designing the “best” architecture remains part art, part science.

2.8 Backpropagation in a Simple CNN

Setup

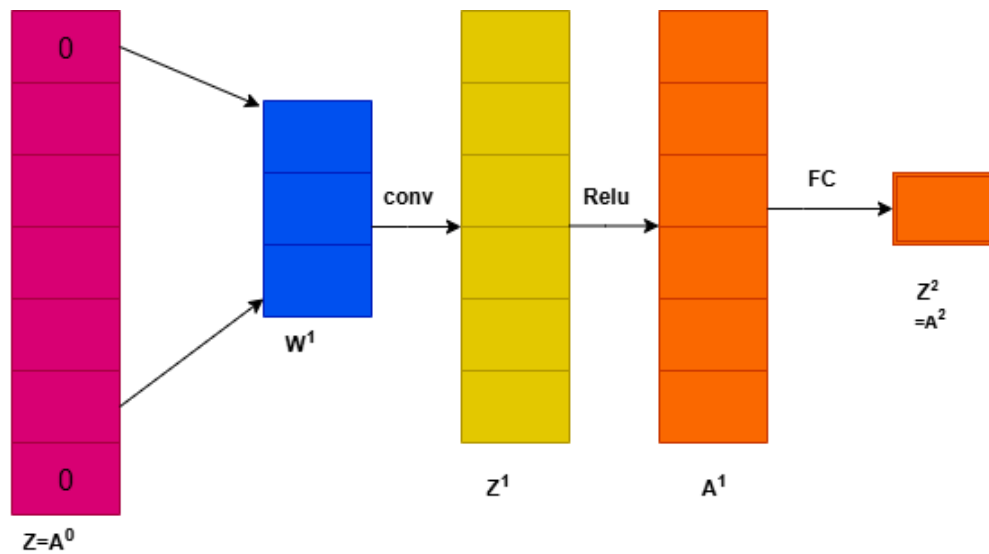


Figure 12: Illustration of backpropagation for a 1D convolutional layer with ReLU activation. Shows dependence of output on filter weights and gradient flow.

We work with a one-dimensional single-channel image, of size $n \times 1 \times 1$. The architecture is:

1. Input image: $X = A^0$.
2. Convolutional layer: a single $k \times 1 \times 1$ filter W^1 .
3. ReLU activation.
4. Fully-connected layer with weights W^2 , no extra activation.
5. Loss: squared error $L(A^2, y) = (A^2 - y)^2$.

Forward Pass

$$\begin{aligned} Z_i^1 &= (W^1)^T \cdot A^0[i - \lfloor k/2 \rfloor : i + \lfloor k/2 \rfloor] \\ A^1 &= \text{ReLU}(Z^1) \\ A^2 &= (W^2)^T A^1 \\ L(A^2, y) &= (A^2 - y)^2 \end{aligned}$$

Backward Pass

Using the chain rule:

$$\frac{\partial L}{\partial W^1} = \frac{\partial Z^1}{\partial W^1} \cdot \frac{\partial A^1}{\partial Z^1} \cdot \frac{\partial L}{\partial A^1}.$$

- **Step 1: Convolution filter dependence**

$$\frac{\partial Z_i^1}{\partial W_j^1} = X_{i - \lfloor k/2 \rfloor + j - 1}.$$

- **Step 2: ReLU derivative**

$$\frac{\partial A_i^1}{\partial Z_i^1} = \begin{cases} 1 & \text{if } Z_i^1 > 0 \\ 0 & \text{otherwise} \end{cases}$$

- **Step 3: Gradient wrt A^1**

$$\frac{\partial L}{\partial A^1} = \frac{\partial L}{\partial A^2} \cdot \frac{\partial A^2}{\partial A^1} = 2(A^2 - y)W^2,$$

an $n \times 1$ vector.

Thus, multiplying these components yields the gradient with respect to W^1 , of shape $k \times 1$.

Worked Example

If $i = 10$ and $k = 5$, then the entries of column 10 in $\frac{\partial Z^1}{\partial W^1}$ are:

$$[X_8, X_9, X_{10}, X_{11}, X_{12}]^T.$$

This shows how pixel 10 of the output depends on the 5 filter weights.

Study Questions

1. For a filter of size k , how much padding do we need to add to the top and bottom of the image?

2. How is the gradient with respect to W^1 assembled from the local dependencies and the ReLU mask?
3. If we used sigmoid instead of ReLU, how would $\frac{\partial A^1}{\partial Z^1}$ change?

Differentiability and Training

The entire architecture can be seen as a single large differentiable function:

$$f(\mathbf{x}; \mathbf{W}) \rightarrow \mathbf{y}$$

where $\mathbf{W}$ are the weights (filters, FC layers). Because the mapping is differentiable, we can compute gradients using backpropagation and optimize the weights using gradient descent.

Study Question

Why do we need non-linear activations (like ReLU) between convolution layers? What would happen if we stacked only convolutional and pooling layers without ReLU?

Training CNNs

- Backpropagation through convolution layers (weight sharing).
- Gradient flow through ReLU and pooling.
- Loss: cross-entropy for classification.
- Optimizers: SGD, Adam.

Hands-on Lab

Build and train a small CNN in PyTorch or TensorFlow for MNIST classification.

3 Case Studies

3.1 Why Look at Case Studies?

In this section, we present several case studies of convolutional neural networks (CNNs). The goal is to give intuition on how to effectively combine convolutional layers, pooling layers, and fully connected layers. Just as many learn coding by reading other people's code, one of the best ways to understand CNN architectures is by seeing examples of effective networks.

A neural network architecture that works well for one computer vision task often transfers to other tasks. For instance, a network trained to recognize cats and dogs may be adapted for self-driving car applications. Later sections will also include references to seminal research papers, helping you understand the foundations of modern CNNs.

3.2 Outline of Case Studies

We will examine:

- Classic networks: **LeNet-5**, **AlexNet**, **VGG**
- Modern networks: **ResNet** (Residual Networks), **Inception**

After studying these examples, you will gain intuition for building effective CNNs, and many ideas from these networks have cross-disciplinary applications beyond computer vision.

3.3 Classic Networks

3.4 LeNet-5

- Input: $32 \times 32 \times 1$ grayscale image (handwritten digits)
- Layer 1: Convolution with 6 filters of size 5×5 , stride 1, output $28 \times 28 \times 6$
- Layer 2: Average pooling, 2×2 filter, stride 2, output $14 \times 14 \times 6$
- Layer 3: Convolution with 16 filters of size 5×5 , output $10 \times 10 \times 16$
- Layer 4: Average pooling, output $5 \times 5 \times 16$ (flattened to 400)
- Layer 5: Fully connected, $400 \rightarrow 120$ neurons
- Layer 6: Fully connected, $120 \rightarrow 84$ neurons
- Output: 10-way classification (digits 0–9)

Notes: Height and width decrease as depth increases. Conv + pooling layers often repeat before fully connected layers. Original LeNet-5 used sigmoid/tanh activations and average pooling; modern variants use ReLU and max pooling.

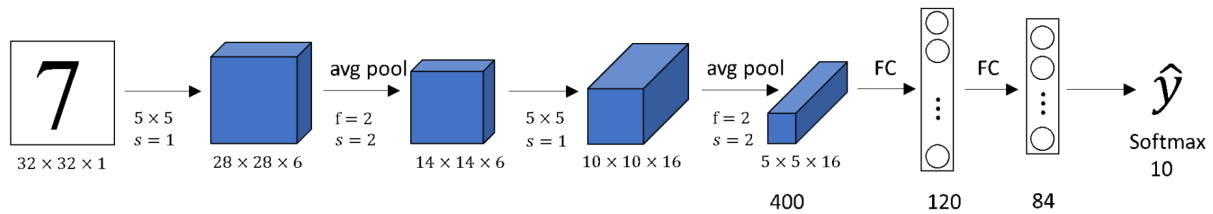


Figure 13: LeNet-5 Architecture. Height/width decrease, channels increase through layers.

3.5 AlexNet: A Turning Point in Deep Learning

3.5.1 The ImageNet Competition

In 2012, the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** became a historic moment for deep learning. The task was to classify over **1.2 million high-resolution images** into **1000 categories**, a scale that challenged all traditional computer vision methods based on hand-engineered features such as SIFT and HOG. see the official ILSVRC website: [ImageNet Large Scale Visual Recognition Challenge \(ILSVRC\)](http://www.image-net.org/challenge/).

AlexNet, developed by *Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton* at the University of Toronto, achieved a **top-5 error rate of 15.3%**, while the runner-up had 26.2%. This dramatic improvement convinced the computer vision community to take deep learning seriously.

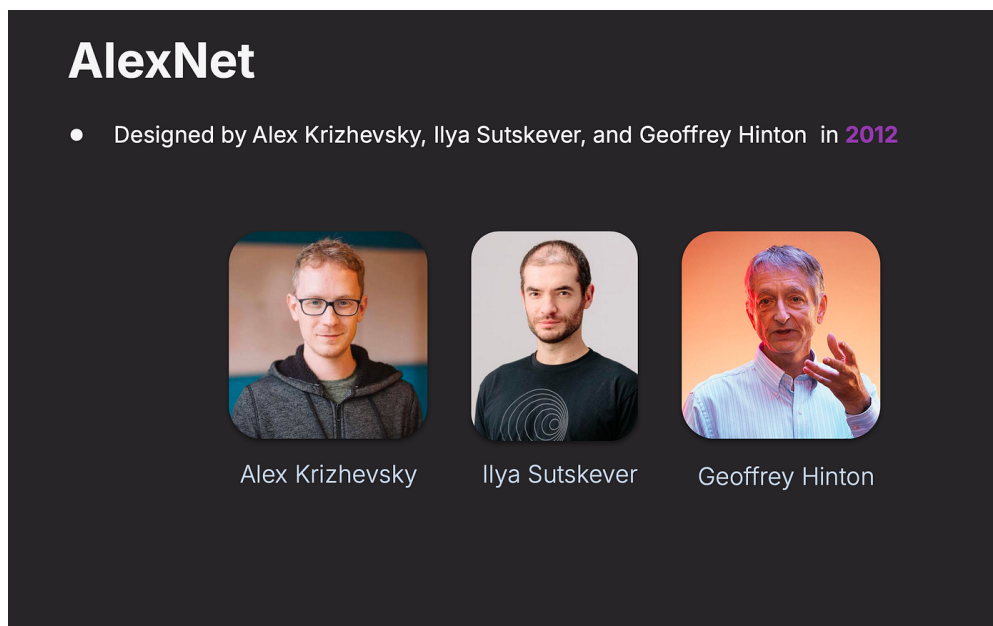


Figure 14: Geoffrey Hinton with Alex Krizhevsky and Ilya Sutskever, the pioneers of AlexNet.

3.5.2 Why AlexNet Was Revolutionary

Several innovations made AlexNet possible:

- **GPUs for Training:** Accelerated learning on large-scale data.
- **ReLU Activation:** Faster training and mitigation of vanishing gradients.
- **Dropout Regularization:** Reduced overfitting in fully connected layers.
- **Local Response Normalization (LRN):** Encouraged competition between neurons (later abandoned in modern architectures).

3.5.3 The AlexNet Architecture

Although AlexNet resembled the earlier LeNet-5, it was significantly larger:

- Input: $227 \times 227 \times 3$ RGB image (paper mentions 224×224 , but 227 fits the stride math).
- Conv1: 96 filters of 11×11 , stride 4 $\rightarrow 55 \times 55 \times 96$.
- Max Pool: 3×3 , stride 2 $\rightarrow 27 \times 27 \times 96$.
- Conv2: 256 filters of 5×5 (same padding) $\rightarrow 27 \times 27 \times 256$.
- Max Pool: 3×3 , stride 2 $\rightarrow 13 \times 13 \times 256$.
- Conv3: 384 filters of 3×3 (same padding) $\rightarrow 13 \times 13 \times 384$.
- Conv4: 384 filters of 3×3 (same padding) $\rightarrow 13 \times 13 \times 384$.
- Conv5: 256 filters of 3×3 (same padding) $\rightarrow 13 \times 13 \times 256$.
- Max Pool: 3×3 , stride 2 $\rightarrow 6 \times 6 \times 256$.
- Flatten: $6 \times 6 \times 256 = 9216$ units.
- Fully Connected Layers: $4096 \rightarrow 4096 \rightarrow 1000$ (softmax).

3.5.4 Impact

- LeNet-5 (1998) had $\sim 60,000$ parameters, while AlexNet had ~ 60 million.
- Its success inspired later architectures such as VGG, GoogLeNet, and ResNet.
- Beyond vision, AlexNet triggered deep learning adoption in NLP, speech recognition, and reinforcement learning.

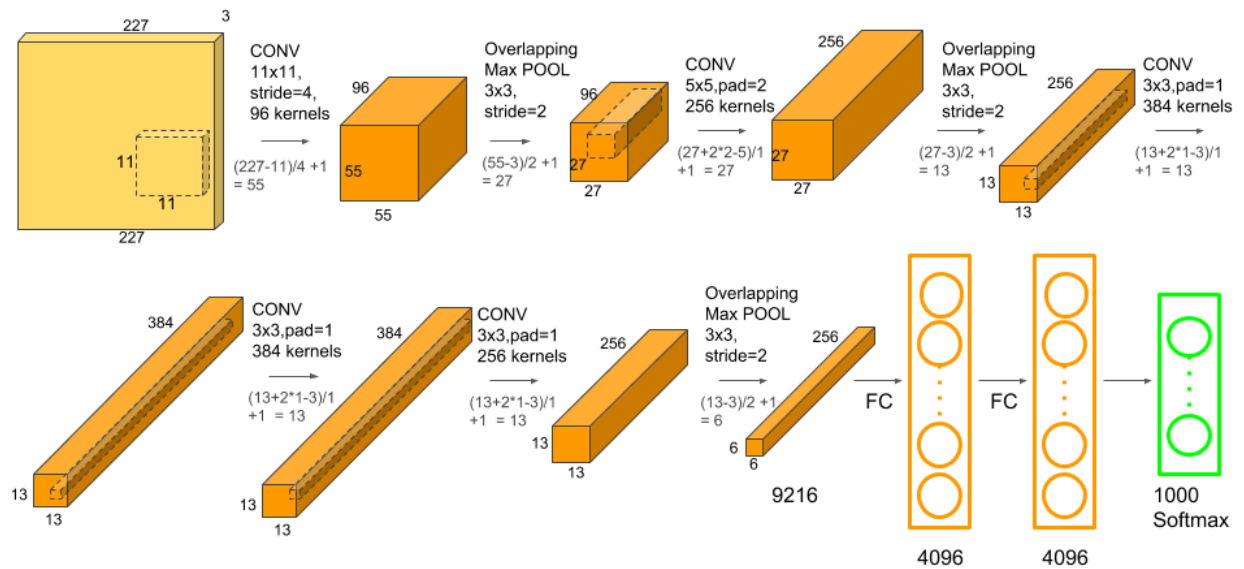


Figure 15: The AlexNet architecture.

3.5.5 Inspiration

AlexNet shows that breakthroughs often come from scaling existing ideas with larger datasets, more powerful hardware, and innovative training methods.

Exercise: Count AlexNet Parameters

Task. Calculate the number of trainable parameters of AlexNet layer by layer. Recall:

- Conv layer: $k_h \times k_w \times C_{in} \times C_{out} + C_{out}$ - FC layer: $N_{in} \times N_{out} + N_{out}$

AlexNet Layers (to use):

1. Conv1: $11 \times 11, 3 \rightarrow 96$
2. Conv2: $5 \times 5, 96 \rightarrow 256$
3. Conv3: $3 \times 3, 256 \rightarrow 384$
4. Conv4: $3 \times 3, 384 \rightarrow 384$
5. Conv5: $3 \times 3, 384 \rightarrow 256$
6. FC1: $9216 \rightarrow 4096$
7. FC2: $4096 \rightarrow 4096$
8. FC3: $4096 \rightarrow 1000$

Fill in the table:

Layer	Parameters
Conv1	_____
Conv2	_____
Conv3	_____
Conv4	_____
Conv5	_____
FC1	_____
FC2	_____
FC3	_____
Total	_____

Hint: Which layers dominate the parameter count? Why?

Reference

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. NIPS. Available at: <http://www.cs.toronto.edu/~fritz/absps/imagenet.pdf>

3.6 VGGNet (VGG-16)

Two years after the breakthrough of AlexNet, the Visual Geometry Group (VGG) at Oxford proposed a simpler but much deeper convolutional neural network architecture. In their 2014 paper, “*Very Deep Convolutional Networks for Large-Scale Image Recognition*” (Simonyan & Zisserman), they demonstrated that increasing network depth, while using very small convolutional filters, could significantly improve performance on the ImageNet dataset.

Key ideas:

- All convolutional layers use small 3×3 filters with stride 1 and padding 1.
- Pooling layers use 2×2 max pooling with stride 2.
- Depth: the most popular versions are VGG-16 and VGG-19, with 16 and 19 weight layers respectively.
- A very large number of parameters (≈ 138 million for VGG-16), making the model computationally expensive.
- Conceptual simplicity: the architecture is just a straightforward stack of convolutions and pooling layers, followed by fully connected layers.

Architecture:

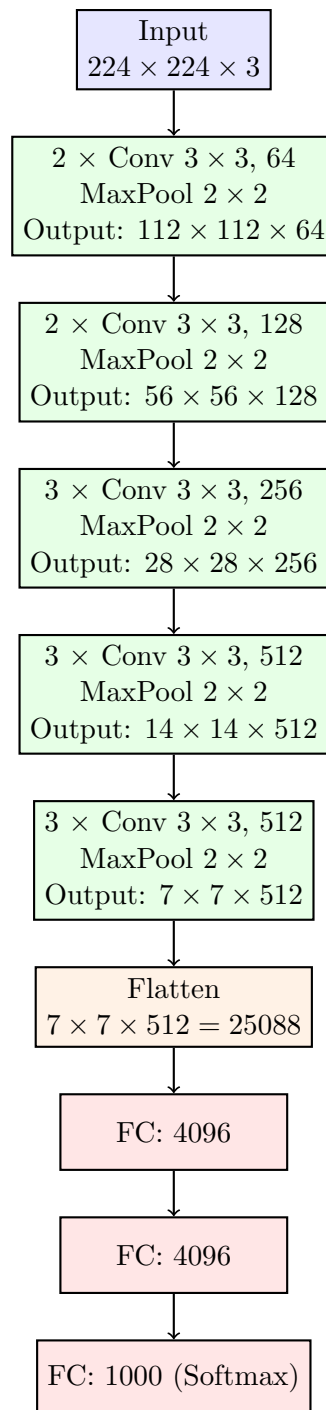


Figure 16: VGG-16 architecture: 13 convolutional layers (3×3 kernels) organized into 5 blocks, followed by 3 fully connected layers.

Why it matters: Despite being heavy in parameters and memory usage, VGG became one of the most influential architectures in computer vision. Its simple design—stacking small filters—established a clear blueprint that influenced many future models. Although

architectures such as GoogLeNet (Inception) performed better in the same ImageNet 2014 competition by being more parameter-efficient, VGG's design had a lasting impact and is still widely used today for transfer learning.

3.7 Residual Networks (ResNets)

*Motivation Very deep neural networks are notoriously difficult to train due to the problems of **vanishing** and **exploding gradients**. In practice, when the number of layers is increased beyond a certain threshold, training error often worsens instead of improving. In theory, adding more layers should increase the representational power of the network; in practice, however, optimization becomes unstable and performance saturates or degrades.

Residual Block

The key idea of ResNet is the **residual block**. Instead of forcing information to flow strictly through the main sequence of linear and non-linear transformations, ResNets introduce a **skip connection** (or shortcut).

Mathematically, for two layers of computation we normally have:

$$a^{[l+2]} = g(z^{[l+2]})$$

where $z^{[l+2]} = W^{[l+2]}a^{[l+1]} + b^{[l+2]}$.

In a residual block, the activation $a^{[l]}$ is added directly to this computation:

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

This skip connection allows the block to learn a residual mapping $F(a^{[l]}) := a^{[l+2]} - a^{[l]}$, making it easier for the network to approximate identity functions if needed. Thus, deeper networks do not necessarily degrade in training performance.

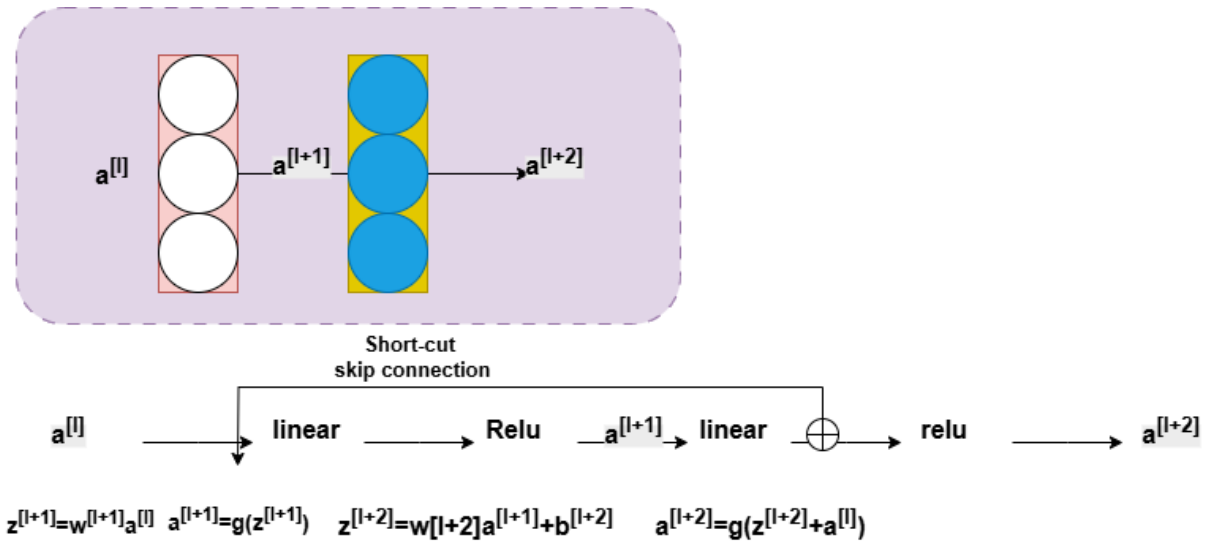


Figure 17: A residual block with a skip connection. Instead of relying solely on the main path, the input activation $a^{[l]}$ is also directly added to the output before the nonlinearity.

Building Residual Networks

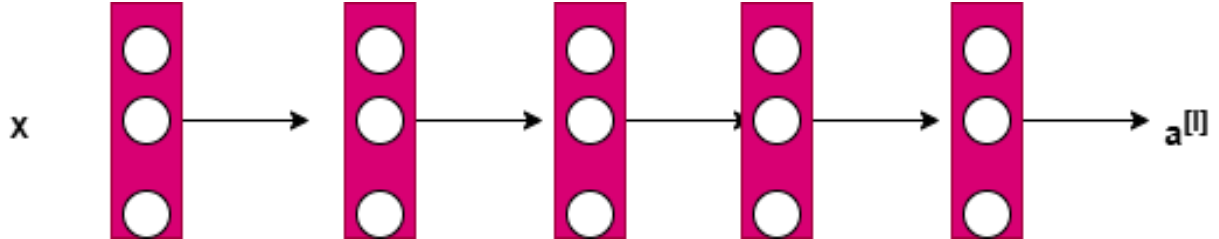


Figure 18: Illustration of the ResNet architecture with stacked residual blocks.

ResNets are constructed by **stacking residual blocks**. For example, the original ResNet-34 is composed of 34 layers of convolution, grouped into stages with shortcut connections linking across two layers at a time.

The figure below contrasts a *plain 34-layer network* with a *34-layer ResNet*. Both have the same number of layers, but the addition of skip connections in ResNet drastically improves trainability.

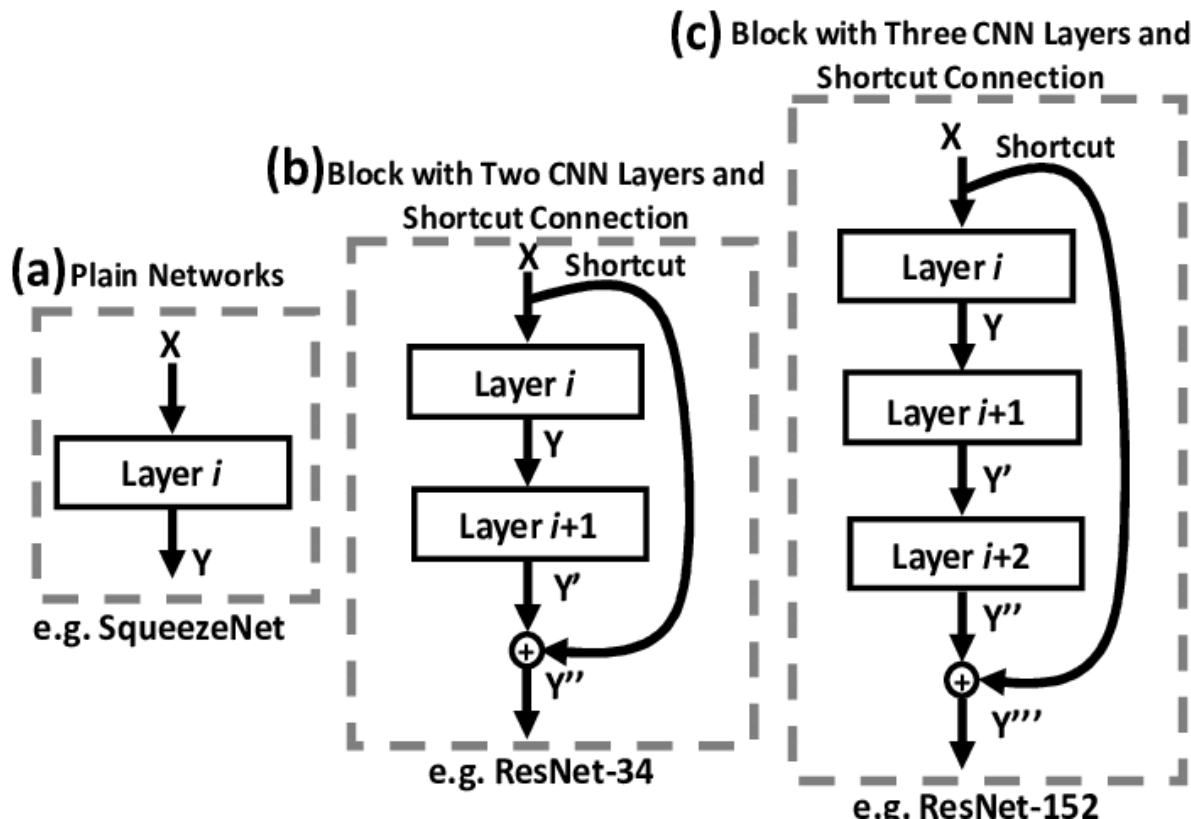


Figure 19: Comparison of a 34-layer plain convolutional network (top) and a 34-layer Residual Network (ResNet) (bottom). In the ResNet, skip connections are added every two layers, enabling much deeper networks to be trained effectively.

Why ResNets Work

The success of ResNets can be attributed to the following:

- **Easier optimization:** residual blocks enable layers to learn small refinements (residuals) rather than the full transformation, making training stable.
- **Identity mapping:** if deeper layers are unnecessary, residual blocks can default to the identity function, ensuring that performance does not degrade.
- **Scalability:** ResNets have been successfully trained with 100, 152, and even 1000+ layers, achieving state-of-the-art results on large-scale vision tasks.

Applications

ResNets form the backbone of many modern computer vision architectures, including detection (Faster R-CNN), segmentation (Mask R-CNN), and beyond. Their success lies in their simple yet powerful design that alleviates vanishing gradients and enables very deep learning.

Active Recall: CNN Architectures

LeNet

Q1: Why is LeNet considered pioneering? **A:** It introduced convolution + pooling + fully connected layers for digit recognition.

Q2: What activation was used in the original LeNet? **A:** Sigmoid/tanh (ReLU came much later).

Q3: What was LeNet's main application? **A:** Handwritten digit recognition on MNIST and bank checks.

AlexNet

Q1: Why did AlexNet win ILSVRC 2012 by a large margin? **A:** Deeper CNN, ReLU activations, dropout, and GPU acceleration.

Q2: What regularization trick did it introduce? **A:** Dropout to reduce overfitting.

Q3: Why was ReLU crucial? **A:** It avoided saturation and sped up convergence.

VGG

Q1: What is the main design principle of VGG? **A:** Use many 3×3 convolutions stacked deep instead of large kernels.

Q2: Why are smaller filters better? **A:** They capture fine features, reduce parameters, and deepen the network.

Q3: What is a limitation of VGG? **A:** High computational cost and memory usage.

GoogLeNet (Inception)

Q1: What problem did Inception modules solve? **A:** How to decide kernel size (1×1 , 3×3 , 5×5) → solution: use all in parallel.

Q2: Why is 1×1 convolution important? **A:** Dimensionality reduction + added nonlinearity.

Q3: How deep was GoogLeNet? **A:** 22 layers with reduced parameters compared to VGG.

ResNet

Q1: Why do residual connections help very deep networks? **A:** They mitigate vanishing gradients and allow identity mapping, enabling training of much deeper networks.

Q2: What happens if a residual block learns zero weights? **A:** The block becomes an identity function, ensuring performance is never worse than a shallower network.

Q3: How are residual blocks typically stacked in ResNet-34? **A:** Skip connections are added every two convolutional layers, grouped into 4 main stages of increasing depth.

4 From Image Classification to Computer Vision Tasks

After gaining a deep understanding of CNN architectures and effective regularization strategies, it is natural to extend our focus to the broader landscape of tasks in deep learning for computer vision. Computer vision (CV) is a field of artificial intelligence that enables machines to interpret, understand, and derive meaningful information from visual data like images or video streams, imitating — and sometimes exceeding — human perception.

At the most fundamental level lies **image classification**, where a single label is assigned to an entire image — the primary focus of our previous chapters. However, many real-world applications require richer and more structured understanding than a single label.

4.1 Image Classification vs Object Detection

Image classification focuses on identifying the overall content of an image (e.g., “cat”, “street scene”, “healthy leaf”), and may be either single-label (one category per image) or multi-label (several categories in one image).

In contrast, **object detection** not only identifies which object types are present but also localizes each instance with bounding boxes, allowing the model to count and locate multiple objects simultaneously.

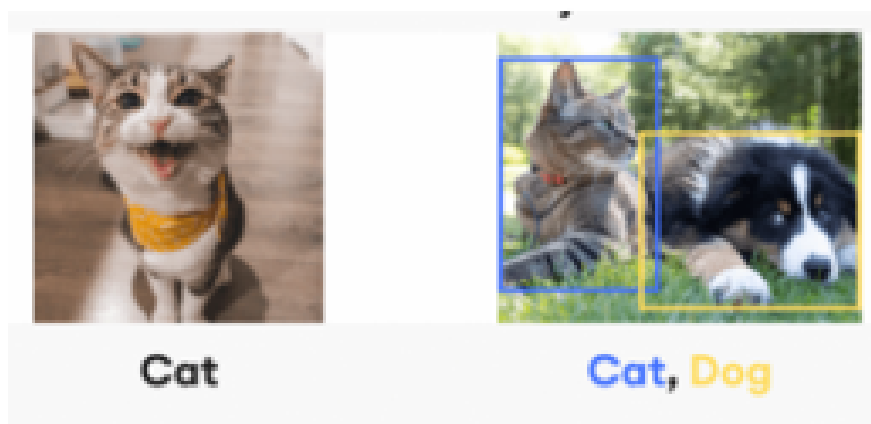


Figure 20: Comparison between image classification (single label) and object detection (bounding boxes). The left side shows a standard classification label, while the right side shows detected objects with bounding boxes.

Use cases: medical diagnosis (classification), autonomous driving (detection of pedestrians, vehicles), retail analytics (detect products on shelves).

4.2 Segmentation

Segmentation divides an image into meaningful regions or objects, assigning a class or label to each pixel. This allows the model to precisely delineate object boundaries and understand the structure of a scene at a fine granularity. Segmentation is widely used in applications such as autonomous driving, medical imaging (e.g., tumor detection), and robotics. Unlike detection, segmentation captures the full shape and area of objects rather than just bounding boxes.



Figure 21: Segmentation: pixel-level labeling of objects in the image.

4.3 Multimodal Understanding

Image captioning generates natural language descriptions of scenes, bridging vision and language. Modern multimodal models like CLIP/GPT-like architectures can understand images and text jointly, enabling richer tasks such as visual question answering and reasoning.

"This is a well-lit kitchen featuring a white farmhouse sink with a matte black faucet under two windows, allowing natural light to flood the space. Wooden countertops and open shelves adorned with glassware and mugs add warmth and character to the room."

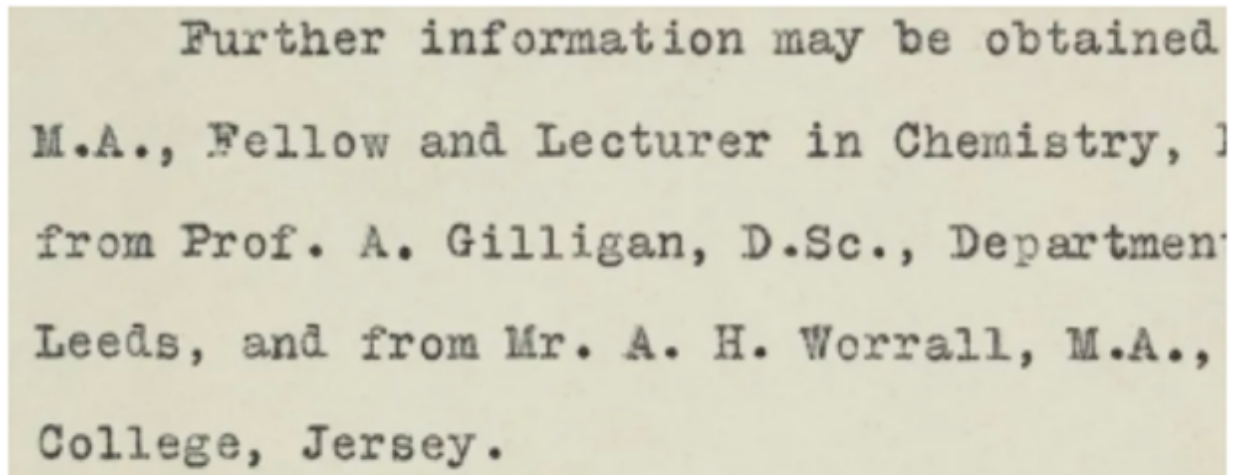


Figure 22: Image captioning: generating text that describes the image content.

Use cases: accessibility tools for visual impairment, automated photo organization, AI assistants that interpret scenes.

Optical Character Recognition (OCR)

OCR detects and extracts text from images or video frames and converts it into machine-readable text, essentially translating visual text into digital data.



Further information may be obtained M.A., Fellow and Lecturer in Chemistry, from Prof. A. Gilligan, D.Sc., Department Leeds, and from Mr. A. H. Worrall, M.A., College, Jersey.

Figure 23: Optical character recognition (OCR): extracting printed or handwritten text.

Use cases: digitizing documents and invoices, automatic number plate recognition, real-time translation of signs.

4.4 Pose Estimation and Keypoint Detection

Pose estimation detects specific anatomical or structural keypoints (e.g., joints on a human body), allowing models to understand orientation, movement, and body structure.



Figure 24: Pose estimation: identifying keypoints such as body joints.

Use cases: sports analytics, physical therapy feedback, gesture control and animation.

4.5 Object Tracking

Object tracking preserves identities of objects across consecutive frames in video, enabling analysis of motion, speed, and trajectories. Unlike detection alone, tracking considers temporal continuity.

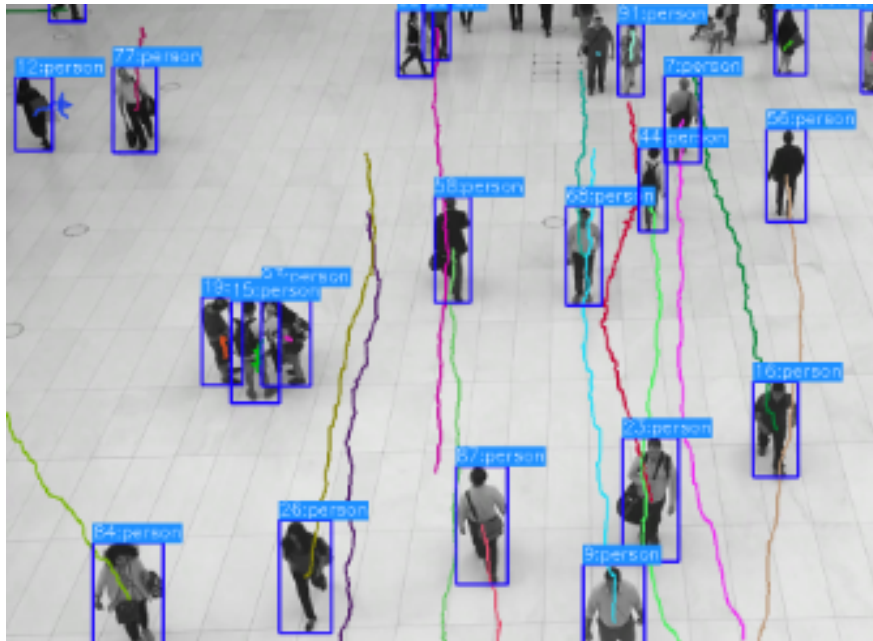


Figure 25: Object tracking: following objects through time in videos.

Use cases: sports analytics, traffic analysis, surveillance monitoring.

4.6 Object Counting

Object counting estimates the number of instances of an object in an image or video. While related to detection, it often requires specialized techniques, especially in dense or occluded scenes.



Figure 26: Object counting: estimating the number of objects in a scene.

Use cases: crowd counting, inventory checks, automated attendance.

Higher-Level and Emerging Tasks

In addition to the fundamentals above, modern computer vision includes:

- **Depth Estimation:** predicting the distance of objects from the camera.
- **Instance Segmentation:** combining detection and segmentation to delineate each object separately.
- **Visual Reasoning / Vision-Language Models:** answering questions about images or integrating vision with other modalities.

Together, these tasks show how modern CNN-based systems evolve from simple recognition toward comprehensive spatial, structural, and temporal scene understanding — enabling real-world applications across healthcare, robotics, autonomous systems, and multimedia.

5 YOLO: You only look once

5.1 Abstract and Introduction: Core Idea

YOLO (You Only Look Once) introduces a new paradigm for object detection by reformulating it as a single regression problem. Unlike previous approaches that repurpose classifiers through sliding windows or region proposal pipelines, YOLO uses one convolutional neural network to directly predict bounding boxes and class probabilities from the entire image in a single forward pass.

This unified, end-to-end architecture eliminates multi-stage processing and enables real-time detection. By reasoning globally over the full image, YOLO incorporates contextual information and reduces background false positives compared to region-based detectors.

The central contribution of YOLO is the simplification of object detection into a single differentiable model, making fast and practical real-time detection possible.

5.2 Unified Detection Formulation

YOLO models object detection as a regression problem. The input image is divided into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose center falls inside the cell.

For each grid cell, the model predicts:

- B bounding boxes,
- A confidence score for each bounding box,
- C conditional class probabilities.

Each bounding box prediction consists of five values: $(x, y, w, h, \text{confidence})$.

The coordinates (x, y) represent the center of the box relative to the grid cell, while the width w and height h are predicted relative to the entire image.

The confidence score reflects both whether an object exists in the box and how well the predicted box overlaps with the ground truth. Formally, the confidence is defined as:

$$\text{Confidence} = \Pr(\text{Object}) \times \text{IOU}_{\text{pred}}^{\text{truth}}.$$

If no object is present in a cell, the confidence score should be zero. Otherwise, it equals the Intersection over Union (IOU) between the predicted and ground truth box.

In addition, each grid cell predicts C conditional class probabilities $\Pr(\text{Class}_i \mid \text{Object})$.

Overall, the final prediction tensor has dimension:

$$S \times S \times (B \times 5 + C).$$

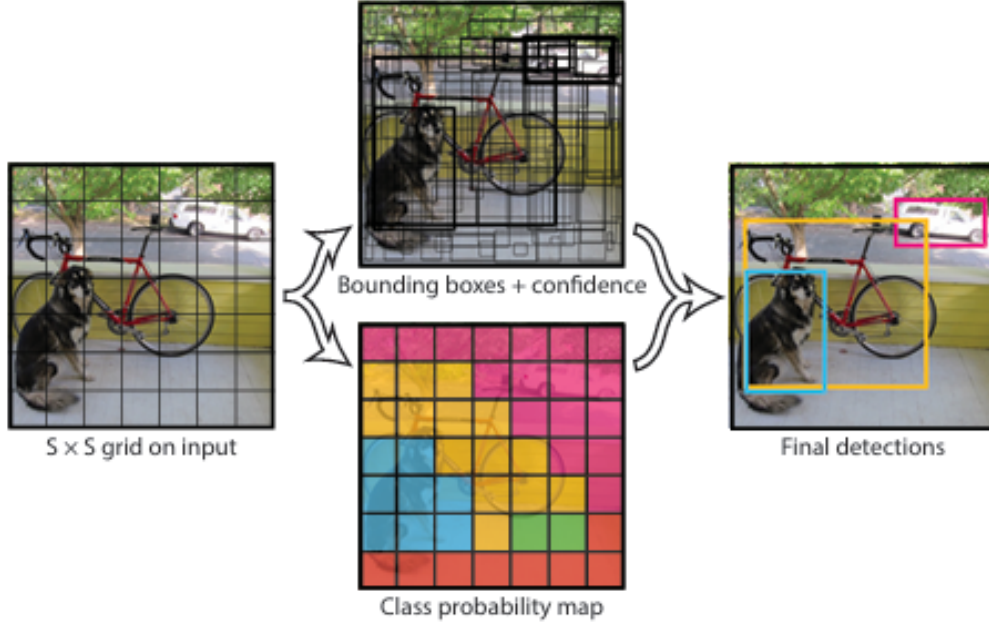


Figure 27: YOLO detection formulation. The image is divided into an $S \times S$ grid. Each grid cell predicts B bounding boxes, confidence scores, and C class probabilities.

5.3 Network Architecture (High-Level)

The YOLO detection network consists of 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutions are used to reduce the dimensionality of feature maps.

The convolutional layers are first pretrained on the ImageNet classification task at lower resolution (224×224). The resolution is then increased for detection (448×448), and the final layers are adapted to predict bounding boxes and class probabilities.

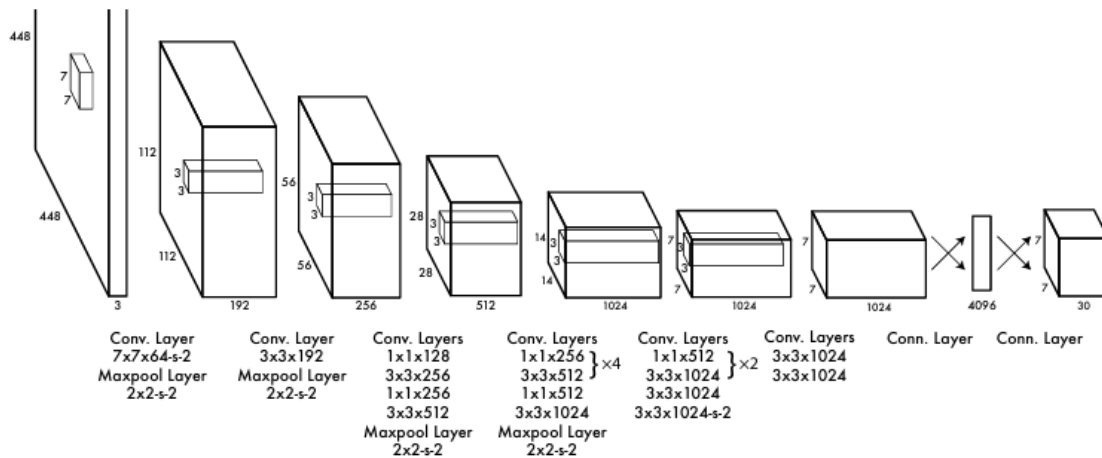


Figure 28: The YOLO architecture. The network consists of convolutional layers followed by fully connected layers that output detection predictions.

5.4 Loss Function

YOLO uses a multi-part loss function that penalizes errors in:

- Bounding box localization (coordinates),
- Bounding box size (width and height),
- Object confidence,
- No-object confidence,
- Classification probabilities.

The complete loss function combines these components:

loss function:

$$\begin{aligned}
& \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}$$

Figure 29: The multi-component YOLO loss function combining localization, confidence, and classification errors.

The localization loss measures errors in predicted box coordinates. The confidence loss penalizes incorrect objectness predictions, with separate weighting for cells containing and not containing objects. The classification loss measures error in predicted class probabilities.

Full mathematical details can be found in the original paper: [“You Only Look Once: Unified, Real-Time Object Detection”](#) (Redmon et al., CVPR 2016).

5.5 Limitations of YOLO

Despite its speed and simplicity, YOLO exhibits several structural limitations.

First, YOLO imposes strong spatial constraints since each grid cell predicts a limited number of bounding boxes and only one class. This restricts the model’s ability to detect multiple nearby objects, particularly small objects appearing in dense groups (e.g., flocks of birds).

Second, because YOLO directly regresses bounding box coordinates from data, it struggles to generalize to objects with unusual aspect ratios or configurations. The use of multiple downsampling layers also leads to coarse feature maps, reducing localization precision for small objects.

Finally, the training loss function does not perfectly align with detection evaluation metrics. Localization errors in small bounding boxes are penalized similarly to those in large boxes,

even though small deviations have a much larger impact on Intersection over Union (IoU). As a result, incorrect localization remains the primary source of error.

6 A Picture is Worth 16×16 Patches: From CNNs to Vision Transformers

Limitations of Convolutional Neural Networks

Convolutional Neural Networks (CNNs) have been the foundation of many computer vision breakthroughs, including image classification, object detection, and segmentation. Despite their success, CNNs have some limitations:

- **Local Receptive Field:** Convolutions process local neighborhoods, requiring many layers to capture global context.
- **Limited Long-Range Dependency:** Relationships between distant regions are captured inefficiently.
- **Fixed Inductive Bias:** CNNs assume locality and translation invariance, which may limit flexibility for diverse data.
- **Scaling Challenges:** Deeper CNNs require careful design, regularization, and computational resources.

6.1 Vision Transformer (ViT) Architecture

The Vision Transformer (ViT), introduced by Dosovitskiy et al. (2020), adapts the Transformer architecture, originally designed for natural language processing, to image data. Unlike CNNs, which rely on convolutional layers to extract local features, ViTs operate on sequences of image patches, allowing them to model long-range dependencies efficiently.

Key components of ViT:

- **Patch Embedding:** The input image is divided into non-overlapping patches (typically 16×16 pixels). Each patch is flattened and projected into a fixed-dimensional embedding vector.
- **Position Embedding:** Since Transformers are permutation-invariant, learnable position embeddings are added to retain spatial information.
- **Transformer Encoder Layers:** Each patch embedding is processed by a stack of standard Transformer encoder blocks, which consist of:
 - Multi-Head Self-Attention (MSA) for capturing global relationships.
 - Feed-Forward Networks (FFN) for non-linear feature transformations.
 - Layer Normalization and residual connections for stable training.

- **Classification Token (CLS):** A learnable [CLS] token is prepended to the patch embeddings and used to summarize the representation for classification tasks.
- **Output Head:** The output corresponding to the [CLS] token is fed into a classifier for final predictions.

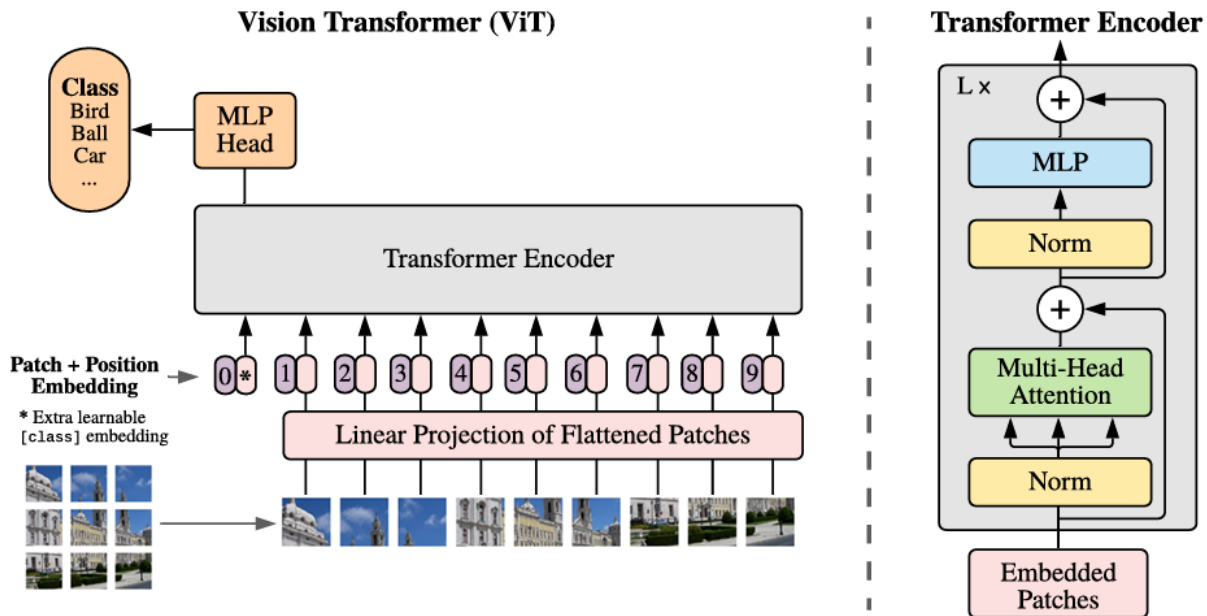


Figure 30: Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

6.2 Comparison Between CNNs and Vision Transformers

The key differences between CNNs and Vision Transformers (ViTs) can be summarized as follows:

Property	CNN	Vision Transformer (ViT)
Feature Extraction	Local features via convolutional filters	Global features via self-attention on image patches
Inductive Bias	Strong (locality, translation invariance)	Weak (learns relationships from data)
Receptive Field	Grows with depth	Global from the first layer
Data Requirement	Works well on small-to-medium datasets	Data-hungry; needs large datasets or pretraining
Scalability	Moderate; deeper networks require careful design	Scales well with model size and dataset size
Applications	Image classification, detection, segmentation	Global context tasks, image classification with large datasets, long-range dependency modeling

Table 1: Comparison of CNN and Vision Transformer (ViT) architectures across key properties.

7 Conclusion

Note

This module provides the foundations of CNNs, from intuition and architecture to real-world applications and projects.

References

- [1] D. H. Hubel and T. N. Wiesel. "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex." *The Journal of Physiology*, 1962.
- [2] K. Fukushima. "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition." *Biological Cybernetics*, 1980.
- [3] Y. LeCun, et al. "Gradient-based learning applied to document recognition." *Proceedings of the IEEE*, 1998.
- [4] A. Krizhevsky, et al. "ImageNet classification with deep convolutional neural networks." *NIPS*, 2012.
- [5] J. Redmon, et al. "You Only Look Once: Unified, Real-Time Object Detection." *CVPR*, 2016.
- [6] A. Dosovitskiy, et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale." *ICLR*, 2021.
- [7] L. P. Kaelbling. "6.036: Introduction to Machine Learning." Massachusetts Institute of Technology (MIT), 2020. Available: <https://ocw.mit.edu>